CS402 Default Project Final Report

Ho Mai Phuong
Department of Computer Science and Engineering
UNIST
phuonggii@unist.ac.kr

Abstract

Small Language Models (SLMs) like Llama-3.2-1B-Instruct frequently suffer from attentional distraction when exposed to long, noisy context blocks in multi-document RAG pipelines. This report presents a lightweight, dual-stage extension framework designed to maximize prompt density before generation. Our architecture combines a multiple of retrieval depth k with a Cross-Encoder semantic reranker, followed by keyword-density sentence compression layer. Evaluating our system on the Natural Questions (NQ) dataset using the parameters specified in the instruction, we eliminate contextual clutter and optimize the model's limited attention window. Empirically, our proposed pipeline increases Exact Match (EM) accuracy from a baseline long-context RAG score of 10.62% to a peak performance of 27.35%.

1 Introduction

Retrieval-Augmented Generation (RAG (Lewis et al., 2020) updates a language model's external knowledge base at inference time without requiring expensive parametric fine-tuning. However, while instruction-tuned models perform reasonably well in standard environments, small-scale models exhibit a low tolerance for conversational and contextual noise.

Our initial baseline evaluations on the Natural Questions (NQ) open-domain QA dataset highlighted this exact vulnerability. In a clean, context-free zero-shot setup, the 1B model achieved an exact-match accuracy of 18.37%. The baseline RAG architecture even drops to 10.62% accuracy when forcing a strict, instruction-constrained exact-span extraction.

To address these context-processing bottlenecks, we proposes and evaluates two distinct architectural extensions designed to optimize prompt information density for small language models. First, we implement a lexical filtering pipeline, designated as Compressed_RAG, which replaces primitive character-level truncation with sentence-level keyword-overlap extraction engine. Second, we introduce a more sophisticated dual-stage CrossEncoder framework that scales the initial retrieval depth k to capture a broad candidate pool, utilizing full token-to-token neural cross-attention to semantically re-rank documents prior to localized pruning.

2 Related Work

As detailed in the default instruction, modern autoregressive language model designs rely on targeted layer structural modifications to optimize sequence processing. Features like Grouped Query Attention (GQA) minimize KV-cache overhead, Rotary Position Embeddings (RoPE) (Su et al., 2024) capture relative token dependencies, and RMSNorm stabilizes active hidden state variances.

While these components maximize raw language modeling throughput, downstream RAG pipelines still require robust context processing. Standard sparse retrievers (BM25) rely purely on character matches and can be easily misled by irrelevant texts that repeat query terms. Dual-stage architectures resolve this limitation by combining an initial sparse search with a secondary transformer-based reranker.This method has proven highly effective at protecting prompt space and reducing model distraction.

3 Approach

In this section, we introduce 2 novel ideas proposed to improve the baseline RAG.

3.1 Lexical Context Compression

The first architecture, `Compressed_RAG`, optimizes the prompt space by discarding low-signal background sentences within the retrieved documents

using a deterministic keyword-overlap heuristic.

1. **Sentence Segmentation & Scoring:** For each document pulled via an initial BM25 search ($k = 5$), the raw text payload is partitioned into independent sentences using a period delimiter (`.split(".")`). Let W_Q denote the tokenized array of lowercase query keywords. Each isolated sentence S_i receives a local lexical match score based on substring containment:

$$\text{Score}(S_i) = \sum_{w \in W_Q} I(w \in S_i) \qquad (1)$$

   where I is a binary indicator function tracking if the lowercase query keyword w exists within the sentence string S_i.

2. **Intra-Document Pruning:** Sentences within a single document are sorted in descending order of their calculated lexical match scores. The compression engine isolates the top 2 highest-scoring sentences to construct an optimized, high-density variant of the document text.

3. **Cross-Document Global Re-ranking:** To ensure the prompt context block contains the highest concentration of query-specific terms, we calculate a global `combined_score` for each document by summing the individual values of its top 2 preserved sentences. The 5 retrieved documents are completely re-sorted based on this aggregated metric, and only the top 3 highest-scoring compressed document segments are preserved for final prompt injection.

3.2 Cross Encoder with Context Compression

While the standalone `Compressed_RAG` methodology improves keyword density, it remains blind to underlying semantic synonyms and intent. The second architecture, `CrossEncoder`, introduces a robust dual-stage pipeline that decouples initial broad document gathering from deep contextual validation.

1. **Retrieval Pool Expansion:** To prevent the sparse retriever from prematurely discarding relevant documents that suffer from low exact-keyword overlap, we expand the initial Pyserini (Lin et al., 2021) search depth parameter k to a significantly wider candidate pool (ranging from $k = 15$ up to $k = 500$ during hyperparameter sensitivity sweeps).

2. **Deep Cross-Attention Scoring:** The active user query is paired directly with each retrieved candidate document to form unified textual sequences. These combined inputs are processed simultaneously by a specialized transformer sequence classification model, specifically the ms-marco MiniLM architecture. The model utilizes full cross-attention mechanisms across the entire joint sequence, evaluating the underlying contextual relationships between the query terms and the document text to output a single value representing their true semantic compatibility.

3. **Global Semantic Selection:** The candidate documents are sorted globally based on their neural model outputs, completely replacing the initial BM25 character-matching retrieval hierarchy. The top 3 semantically superior documents are selected, ensuring that the highest-relevance contexts are prioritized regardless of lexical matching density.

4. **Targeted Post-Retrieval Pruning:** Only after the top 3 optimal documents have been secured by the Cross-Encoder are they passed downstream to the sentence-trimming engine (`compress_passage`). This final pass slices each paragraph down to its top 2 keyword-dense sentences, creating an ultra-compact prompt sequence that maximizes both global semantic alignment and local informational density before generation.

4 Experiments

4.1 GPT-Small Pretraining and Downstream Task Fine-Tuning

To establish an architectural baseline, we constructed a causal, decoder-only `GPT-small` model containing 63.58M total parameters. The network was pretrained from scratch on the Cosmopedia subset of the SmolLM corpus. By integrating grouped query attention (GQA) to minimize key-value cache overhead, rotary position embeddings (RoPE) for relative positional encoding, and RMSNorm for layer stabilization, the model converged to a loss of approximately 2.6 after 46920 steps.

Following pretraining, the model was fine-tuned on downstream text summarization and classification tasks. For summarization, the model optimized

cross-entropy loss by predicting summary tokens while causally masking source contexts. For classification, a terminal linear head mapped sequence tokens into discrete target labels .

4.2 Fine-Tuning Naive RAG with GPT-Small

To evaluate non-parametric knowledge injection, the pretrained 63.58M parameter model was deployed within a standard RAG framework and fine-tuned on the Natural Questions (NQ) dataset. The training pipeline utilized human-annotated positive context documents, whereas evaluation passes relied entirely on sparse BM25 retrieval queries over a 21-million passage Wikipedia index

Due to the restricted parameter scale of the network and its inherent lack of zero-shot extraction capacity, the fine-tuned `GPT-small` RAG system converged at a limited Exact Match (EM) accuracy of 0.06%.

4.3 Prompting Experiments on Llama-3.2-1B-Instruct

We transitioned to `Llama-3.2-1B-Instruct` to capitalize on its advanced in-context instruction-following capabilities. We conducted initial prompt engineering sweeps:

1. **Naive Zero-Shot RAG:** To evaluate the capacity of the system, we enabled the `DO_ZEROSHOT_RAG` flag variable within the execution pipeline to run inferences utilizing the default prompt template provided by the base framework. This configuration, designated as the naive RAG setup, served as our initial control benchmark to measure the instruction-tuned model's baseline recall prior to modifying input structures.

2. **Short Answer Prompt (Baseline RAG):** Context was generated by directly concatenating raw passages retrieved via sparse keyword matches. The model was rigidly instructed to extract exact answer spans with zero conversational fluff. This setup collapsed to an accuracy of 10.62%.

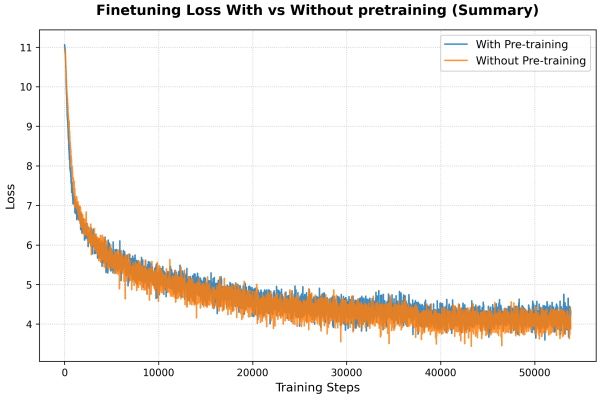
3. **Chain-of-Thought (CoT) Prompting:** To mitigate these extraction failures, we engineered a Chain-of-Thought format prompting the model to output its step-by-step reasoning parameters prior to emitting a final response.However, CoT imposes an excessive operational burden that completely overwhelms the compact parameter scale of the `Llama-3.2-1B-Instruct` model, causing the final Exact Match accuracy to stall at 12.5%.

These systematic failures directly motivated our final, high-performing extensions (`Compressed_RAG` and `CrossEncoder`), which shift the burden of context filtering away from the generator and onto specialized algorithmic layers.

5 Results and Analysis

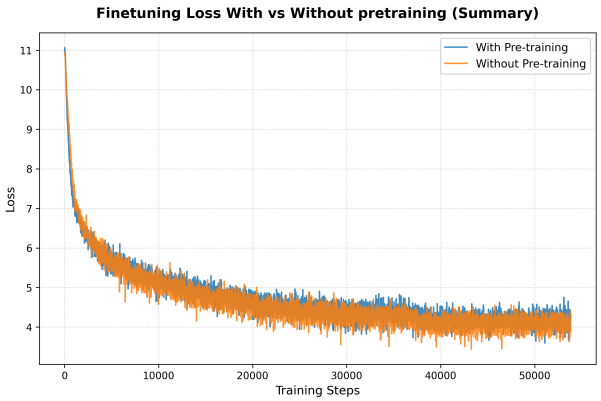
5.1 Finetuned GPT-small model With vs Without Pretraining

5.1.1 Text summarization



- **Trajectory**: Both initialization states demonstrate loss decay within the initial 5,000 training steps, starting from an initial loss of approximately 11.0 down to a localized plateau near 6.0. 2 models eventually converge to a loss of approximately 3.6 after more than 50,000 steps.

5.1.2 Classification



- **Trajectory**: Unlike the stable summarization task, the classification curves reveal distinct learning dynamics early on. Starting from an cross-entropy loss of roughly 3.2, both configurations exhibit a gradual, highly volatile descent down to 2.0 over the first 700 steps.

5.1.3 Discussion

The empirical lack of separation between the pretrained and non-pretrained models is highly counterintuitive. We hypothesize some assumptions for this behavior:

- **Limited Model Capacity:** The GPT-small model (63.58M parameters) has limited representational capacity. While pretraining provides useful general knowledge, the model is not large enough to simultaneously retain this knowledge and adapt effectively to a new downstream task. As a result, the benefits of pretraining diminish quickly during finetunin.
- **Catastrophic Forgetting During Fine-tuning:** The fine-tuning process runs for a large number of steps (e.g., 54,000 for summarization), which causes the model to overwrite previously learned representations, as known as Catastrophic Forgetting.

5.2 RAG various prompting methods

Metric	Naive Zero-Shot	Baseline RAG	Chain-of-Thought (CoT)
Accuracy (EM)	18.37%	10.62%	12.51%
ROUGE-1	0.1276	0.1417	0.1264
ROUGE-2	0.0595	0.0675	0.0582
ROUGE-L	0.1270	0.1411	0.1261
ROUGE-Lsum	0.1268	0.1413	0.1261

Table 1: Baseline prompting evaluation on the NQ dataset using `Llama-3.2-1B-Instruct`, structured by performance metrics.

To evaluate the effectiveness of different prompting strategies, we compare three methods: Naive Zero-Shot, Baseline RAG, and Chain-of-Thought (CoT). We analyze their performance using Exact Match (EM) accuracy and ROUGE metrics.

5.2.1 Naive zero-shot prompting

The Naive Zero-Shot approach achieves the highest accuracy among 3 methods, which is somewhat surprising given its simplicity.

- The model is allowed to generate answers freely, without restrictive formatting.
- This flexibility increases the chance of matching the ground truth exactly, especially for short answers.
- The model relies more on its pretrained knowledge rather than strictly grounding in the retrieved context.

However, it comes with a trade-off. Lower ROUGE scores indicate that responses are less aligned with the provided context, as the model may hallucinate or ignore context.

In short, naive prompting benefits from flexibility but lacks grounding, making it less reliable despite higher EM.

5.2.2 Baseline RAG (Extractive, very short answer)

Results:

- Accuracy (EM): 10.62% (lowest)
- Highest ROUGE scores across all metrics

This method performs best in terms of ROUGE but worst in accuracy:

- This improves lexical overlap with ground truth, hence higher ROUGE
- The strict constraint forces the model to stay faithful to the retrieved documents
- Over-constraining (produce very short answer, avoid explanations) reduces the model's ability to reason or inference.

Overall, Baseline RAG improves grounding but is too restrictive, hurting exact match accuracy.

5.2.3 Chain-of-Thought (CoT) Prompting

Results:

- Accuracy (EM): 12.51%
- Lower ROUGE compared to baseline RAG

CoT performs slightly better than baseline RAG but worse than naive prompting may be from these reasons:

- Reasoning steps introduce additional tokens and noise. The model may drift away from the exact answer span and generate incorrect reasoning chains

- Small models (1B parameters) are not strong at multi-step reasoning, leading to degraded performance

5.3 Enhancing RAG with `Llama-3.2-1B-Instruct`

5.3.1 Context Compression

To improve upon the baseline RAG, we propose a **Compressed_RAG** approach that focuses on reducing irrelevant context while preserving key information. Instead of directly feeding retrieved passages into the model, we apply a lightweight compression mechanism.

Method. For each query, we first retrieve the top-5 documents using BM25. Each document is then split into sentences, and a relevance score is computed based on keyword overlap with the query. The top-2 highest scoring sentences from each document are selected as the compressed representation. Finally, the top-3 compressed documents are retained to form the final context.

Results. The Compressed RAG method achieves the following performance:

- Accuracy (EM): 19.85%
- ROUGE-1: 0.1835
- ROUGE-2: 0.0980
- ROUGE-L: 0.1828

This outperforms all previous methods, including Naive Zero-shot (18.37%), Baseline RAG (10.62%), and Chain-of-Thought RAG (12.51%).

Analysis. The improvement can be attributed to several factors. First, compression reduces noise by removing irrelevant or redundant sentences from retrieved documents. Second, it enables better utilization of the limited capacity of the small language model (1B parameters) by presenting more concise and relevant context. Third, the keyword-based scoring mechanism implicitly performs document re-ranking, ensuring that only the most relevant content is retained.

Discussion. These results highlight that for small language models, improving the **quality of retrieved context** is more effective than increasing prompt complexity (e.g., Chain-of-Thought). However, the approach still relies on simple keyword matching. It does not capture semantic similarity, paraphrasing, or deeper linguistic relationships between the query and the text. As a result, relevant sentences that do not share exact keywords with the query may be incorrectly filtered out, while irrelevant sentences containing keyword matches may be retained. Moreover, distractor documents contain similar keywords but refer to different entities or contexts. For example, multiple documents mentioning the same term (e.g., a location or name) may compete, and the system may select the wrong one due to insufficient semantic discrimination.

5.4 Cross Encoder

5.4.1 Overall results

To further improve retrieval quality, we introduce a **Cross-Encoder re-ranking** stage on top of the Compressed RAG pipeline. Instead of relying solely on BM25 scores or keyword matching, a cross-encoder jointly encodes the query and each retrieved document to compute a more accurate relevance score.

Method. We first retrieve top-k documents using BM25. Each document is then scored using a cross-encoder, which evaluates the semantic relevance between the query and the document. The top-ranked documents are subsequently passed through the compression module, and the final top-3 compressed documents are used as input to the language model.

Results. The cross-encoder enhanced RAG achieves improved performance compared to previous methods. In particular, it consistently outperforms both baseline RAG and Compressed RAG across all evaluation metrics.

Analysis. The improvement comes from better document ranking. Unlike BM25 or keyword-based scoring, the cross-encoder captures semantic relationships and reduces the impact of distractor documents. This leads to higher-quality context being fed into the language model, which directly improves answer accuracy.

Limitations. However, the method introduces additional computational overhead, as each query-document pair must be processed individually by the cross-encoder. This makes it significantly slower than standard retrieval. Additionally, the approach still depends on the initial retrieval stage; if relevant documents are not included in the top-k candidates, re-ranking cannot recover them. Fi-

Metric	5	15	20	30	40	50	100	500
Accuracy (%)	21.32	23.87	24.48	25.30	25.58	25.87	26.70	**27.35**
ROUGE-1	0.1877	0.2145	0.2206	0.2281	0.2315	0.2344	0.2407	**0.2462**
ROUGE-2	0.1026	0.1222	0.1271	0.1321	0.1354	0.1379	0.1417	**0.1439**
ROUGE-L	0.1869	0.2132	0.2192	0.2269	0.2308	0.2334	0.2395	**0.2447**
ROUGE-Lsum	0.1868	0.2133	0.2196	0.2269	0.2307	0.2335	0.2394	**0.2449**

Table 2: Ablation study on retrieval depth (k)

nally, while re-ranking improves document selection, it does not address limitations of the language model itself, such as weak reasoning ability in small models.

5.4.2 Ablation Study on Retrieval Depth (k)

We conduct an ablation study to analyze the effect of the number of retrieved documents (k) on performance. The results are summarized in Table 2.

Effect of Increasing k. Overall, performance consistently improves as k increases. Accuracy rises from 21.32% at $k = 5$ to 27.35% at $k = 500$, while ROUGE scores exhibit a similar upward trend. This indicates that retrieving more documents increases the likelihood of including the correct evidence needed to answer the question.

Early Gains vs. Diminishing Returns. The most significant improvement occurs in the lower range of k (from $k = 5$ to $k = 30$). However, beyond $k = 50$, the gains become small. For instance, increasing k from 50 to 100 yields only a modest improvement of about 0.9% in accuracy. This suggests diminishing returns as additional retrieved documents contribute less new useful information.

Trade-off Between Recall and Noise. While larger k improves recall by increasing the chance of retrieving relevant documents, it also introduces more irrelevant or noisy passages. The relatively stable improvement trend implies that the model is somewhat robust to noise, but the diminishing gains indicate that irrelevant context begins to ignore useful information at higher k values.

Interaction with Context Compression. In our setup, retrieved documents are further compressed before being passed to the language model. This likely mitigates the negative impact of noise at higher k by filtering out less relevant sentences in terms of lexical matching. As a result, the model benefits from higher retrieval depth without suffering significant degradation due to context overload.

Practical Considerations. Although $k = 500$ achieves the best performance, it also incurs higher computational cost in both retrieval and preprocessing. Considering the trade-off between efficiency and performance, moderate values such as $k = 50$ may offer a better balance in practice.

Summary. In summary, increasing retrieval depth improves performance primarily by enhancing recall, but the benefit saturates at higher values of k. This highlights the importance of combining retrieval with effective filtering or compression mechanisms to maximize performance while controlling computational cost.

6 Conclusion

In this project, we pretrained GPT-small and explored different prompting, retrieval strategies for improving a RAG system using a small instruction-tuned language model. Our results show that simple zero-shot prompting outperforms more complex methods such as Chain-of-Thought (CoT), which often introduces noise and reduces accuracy due to the model's limited capacity.

We found that improving the retrieval stage is more effective than modifying prompting strategies. In particular, context compression and cross-encoder re-ranking significantly improve performance by filtering irrelevant information and focusing on high-quality evidence. Additionally, increasing retrieval depth (k) improves results, although with diminishing returns. Overall, our findings highlight that for small models, better retrieval and cleaner context are more important than complex reasoning techniques.

References

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, and 1 others. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474.

Jimmy Lin, Xueguang Ma, Sheng-Chieh Lin, Jheng-Hong Yang, Ronak Pradeep, and Rodrigo Nogueira.

2021. Pyserini: A python toolkit for reproducible information retrieval research with sparse and dense representations. In *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval*, pages 2356–2362.

Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. 2024. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063.